Lab1 CS2

As you have learned in CS1 that heap data structure can be used to maintain a priority queue. The run-time for both insertion and deletion for a single item is O(log n). In a min heap the smallest number will be in the root of the tree and in a maxheap the largest number will be at the root of the tree. So, if you remove something from minhip, you will get the smallest number and for a maxheap you will get the max number from the heap.

Unlike CS1, you really don't have to implement your own heap. Java has a PriorityQueue class that is actually an implementation of minheap. If you want to use it for maxheap you just need to pass `Collections.reverseOrder()` **while instantiating the PrirityQueue.**

For the most part in a priority queue you will be working with four main methods:

- **q.add(newValue);**
- **q.poll();**
- **q.peek();**
- **q.size();**
- **Add: The add method for a priority queue behaves in $O(\log n)$ runtime. The n in this runtime is the number of elements in the priority queue at the time of addition. Here log means log base 2. If you plug 1,000,000 in to the log function of your calculator, you will see this is not a large number of steps. This makes priority queues a nice choice for speeding up problems where you need to know either the largest or smallest element you would like to process next.**
- **The purpose of this method is to add an element to the container. Note that the runtime of $O(\log n)$ is worse than the runtime $O(1)$. Essentally we are paying for the fast lookup of the smallest element by making adding elements slower. Sometimes this is a worthy tradeoff.**
- **Poll: This method takes the smallest element in the queue based on each objects compareTo method and removes it from the queue. The method call takes $O(\log n)$ time as well as the structure must rearrange itself to figure out the next smallest element.**
- **Peek: This method allows you access to the smallest element in the queue without removing it. It works the same as ArrayDeque's peek method and even runs in $O(1)$ time!**

The following lab problem can be solved using priority queue efficiently.

Problem:

- **In this problem you want to read in a list of integer values. After reading in each value print the median of the list so far. There can be up to 10^5 values.**

What is Median?

Median can be defined as the element in the data set which separates the higher half of the data sample from the lower half. When the input size is odd, we take the middle element of sorted data as the median. If the input size is even, we pick an average of middle two elements in the sorted stream as the median.

`Sample Input/Output 1:`

- `Input: 3  //this number represents how many inputs`
  `5 10 15`
- `Output:`
  `5.0`
  `7.5`
  `10.0`

Explanation of the input/output1:

- **as the first input is 5, the output is also 5 as it is the only median**
- **When you got 10, the median is 7.5 as we have even number of items, so we take the average of the middle numbers (5+10)/2 = 7.5**
- **When we got 15, we have odd number of items, so the output will be the middle number in the list which is 10**

Sample input/Output 2:

Input: 4 //this number represents how many inputs
1 2 3 4
Output:
1.0
1.5
2.0
2.5

How to solve this problem. Let's think a bit first.

- **You need to keep track all the numbers coming to you one by one**
- **Whenever you receive the first number, that is itself a median**

- **As new numbers are coming one after another, we need to evaluate median after receiving each number**
- **most importantly you need to know which number is in the middle if the total number of items are now odd.**
- **However, if the total number of items is even, you need to know the two numbers in the middle.**
- **So, conceptually, you can think you have two hands**
- **Left hand will keep half smaller number where the top number in the left hand will be the max among them (maintain a Max heap on the left side!)**
- **The right hand will keep half of the bigger number where the top most number will be the smallest number among those bigger number (Maintain a minheap on the rightside!)**
- **So, in this way, the mid numbers will be always either in the top of left hand and top of the right hand**
- **So, now you need to find a strategy to calculate median when numbers are coming one after another**
- **As we are interested to keep the middle numbers on the top of the heaps, their size should not differ by more than 1.**

Here is the main steps:

1. Create two heaps. One max heap to maintain elements of lower half and one min heap to maintain elements of higher half at any point of time..
2. Take initial value of median as 0.
3. For every newly read element, insert it into either max heap or min-heap and calculate the median based on the following conditions:
    - If the size of max heap is greater than the size of min-heap and the element is less than the previous median then pop the top element from max heap and insert into min-heap and insert the new element to max heap else insert the new element to min-heap. Calculate the new median as the average of top of elements of both max and min heap.
    - If the size of max heap is less than the size of min-heap and the element is greater than the previous median then pop the top element from min-heap and insert into the max heap and insert the new element to min heap else insert the new element to the max heap. Calculate the new median as the average of top of elements of both max and min heap.
    - If the size of both heaps is the same. Then check if the current is less than the previous median or not. If the current element is less than the previous median then insert it to the max heap and a new median will be equal to the top element of max heap. If the current element is greater than the previous median then insert it to min-heap and new median will be equal to the top element of min heap.

Please complete this problem with the help of the TA and submit it in CodeGrade. The file name has to be Main.java